Об Академии

1 80% практика
20% теория

2 Портфолио уже
во время учёбы

3 Индивидуальный
подход

4 Разбор реальных
задач

5 Преподаватели
— действующие
специалисты индустрии

6 Актуальные программы
под современный
IT-рынок

25
Стран

68 000
Студентов

267 000
Выпускников

116
Филиалов

2 900
Преподавателей

Откройте камеру телефона на iPhone
или приложение «Сканер» на Android

Наведите на QR-код

Нажмите на появившуюся ссылку

Выберите ваш город

ШАГ компьютерная АКАДЕМИЯ

Программа курса

ВАЙБ-КОДИНГ

СОЗДАНИЕ FULL STACK ПРИЛОЖЕНИЙ С ИСПОЛЬЗОВАНИЕМ AI TOOLS



ПО ОКОНЧАНИИ КУРСА СЛУШАТЕЛЬ БУДЕТ:

- Понимать основы Artificial Intelligence, Generative AI, LLM и Prompt Engineering
- Понимать, что такое Vibe Coding, где он полезен и какие у него ограничения
- Уметь использовать ChatGPT и Gemini для генерации идей, проработки требований и улучшения prompt"ов
- Уметь использовать Cursor и GitHub Copilot для создания, изменения и сопровождения кода
- Понимать устройство различных типов приложений
- Понимать базовые конструкции HTML, CSS и JavaScript для чтения и правки AI-generated кода
- Уметь использовать Git и GitHub для фиксации этапов работы и безопасной итерации
- Уметь формировать MVP, user flow, feature list и acceptance criteria для приложений, создаваемых через Vibe Coding
- Создавать front-end, back-end и database-слой full stack приложения
- Выполнять интеграцию между клиентом, сервером и базой данных
- Выполнять отладку, тестирование и рефакторинг AI-generated решений
- Понимать подходы к deployment и публикации приложений
- Использовать возможности Claude Code и Codex в AI-assisted и agentic development

Введение в Artificial Intelligence и цифровые продукты

- Что такое Artificial Intelligence
- Чем AI отличается от обычной программы
- Основные направления AI
- Где AI используется в современном software development
- Роль AI в создании цифровых продуктов
- Ограничения AI как инструмента
- Базовые термины курса
- Практические примеры использования

Generative AI, LLM и принципы работы современных моделей

- Что такое Generative AI
- Что такое Large Language Model
- Как модель работает на базовом уровне
- Почему модель может ошибаться
- Галлюцинации и уверенные неверные ответы
- Температура, контекст и влияние формулировки запроса
- Сильные и слабые стороны LLM
- Практические примеры использования

Практические примеры использования

- Что такое Prompt Engineering
- Структура хорошего prompt'а
- Zero-shot, one-shot и few-shot подходы
- Итеративное улучшение prompt'а
- Плохие и хорошие prompt'ы
- Prompt'ы для объяснения
- Prompt'ы для генерации структуры решения
- Prompt'ы для кода, анализа и исправлений
- Практические примеры использования

Проверка, этика, безопасность и ответственная работа с AI

- Почему AI-ответ нужно проверять
- Приёмы проверки результата
- Этические аспекты использования AI
- Приватность данных и чувствительная информация
- Роль AI в обучении и самостоятельной работе студента
- Типичные ошибки новичка при работе с AI
- Практические примеры использования

Что такое Vibe Coding и как он отличается от классической разработки

- Понятие Vibe Coding
- Эволюция подходов к созданию приложений
- Базовые принципы Vibe Coding
- Преимущества подхода
- Ограничения и риски
- Когда Vibe Coding подходит лучше всего
- Практические примеры использования

Обзор инструментов Vibe Coding и карта современного AI workflow

- Инструменты для генерации идей
- Инструменты для AI coding
- AI builders для быстрого создания приложений
- Инструменты следующего уровня
- Как выбирать инструмент под задачу
- Типовой workflow создание продукта с vibe coding
- Практические примеры использования

Из чего состоят разные типы приложений

- Какие типы приложений бывают?
- Разные типы приложений и их особенности
- Типовые пользовательские сценарии
- Практические примеры использования

Архитектура web application: browser, front-end, back-end, database, deployment

- Как работает web application
- Роль browser и client-side логики
- Роль server-side логики
- Роль базы данных
- Что такое API
- Поток данных в приложении
- Где возникают ошибки и узкие места
- Практические примеры использования

Основы HTML для понимания структуры AI-generated code

- Что такое HTML
- Структура HTML-документа
- Базовые теги
- Блочные и строчные элементы
- Семантические элементы
- Формы в HTML
- Атрибуты и структура документа
- Чтение и изменение AI-generated HTML
- Практические примеры использования

Основы CSS для понимания оформления интерфейса

- Что такое CSS
- Способы подключения CSS
- Селекторы
- Цвета, шрифты и текст
- Отступы, границы и размеры
- Box model
- Flexbox
- Grid на базовом уровне
- Практические примеры использования

Основы JavaScript для понимания логики приложения

- Что такое JavaScript
- Переменные и типы данных
- Операторы
- Условные конструкции
- Циклы
- Функции
- Массивы
- Объекты
- Работа со строками и числами
- Чтение и объяснение AI-generated JavaScript
- Практические примеры использования

JavaScript в интерфейсе: DOM, события, формы и хранение данных

- Что такое DOM
- Поиск элементов на странице
- Изменение текста, классов и атрибутов
- События
- Формы и валидация
- Рендеринг списков
- Local state на базовом уровне
- LocalStorage
- Обработка пользовательского ввода
- Практические примеры использования

Чтение, объяснение и точечная правка AI-generated code

- Как читать незнакомый AI-generated code
- Как понимать структуру простого проекта
- Как просить AI объяснить конкретный фрагмент кода
- Как искать файл, отвечающий за нужную часть приложения
- Как находить связь между экраном, компонентом и логикой
- Как вносить маленькие изменения без поломки проекта
- Как просить AI изменить только нужный фрагмент кода
- Как проверять результат маленького изменения через запуск приложения
- Практические примеры использования

Основы Git для безопасного Vibe Coding

- Что такое Version Control
- Зачем Git нужен для Vibe Coding
- Repository, commit, history
- Инициализация репозитория
- Сохранение рабочего состояния
- Просмотр изменений
- Возврат к предыдущему состоянию на базовом уровне
- Практические примеры использования

GitHub и базовый workflow студента

- GitHub как платформа для хранения проекта
- Clone, push, pull
- Branch на базовом уровне
- README
- Фиксация этапов разработки
- Базовый workflow индивидуального проекта
- Использование GitHub совместно с Cursor и GitHub Copilot
- Практические примеры использования

Качество и архитектура AI-generated кода

- Почему AI-generated code требует контроля качества
- Типичные проблемы AI-generated кода
- Single Responsibility Principle
- MVC как простая модель архитектуры приложения
- Простая структура full stack проекта
- Code Review AI-generated кода
- Git diff как инструмент контроля изменений
- Безопасный workflow vibe-coding
- Практические примеры использования

Claude Code и Codex: следующий уровень AI-assisted и agentic development

- Что такое Claude Code?
- Что такое Codex?
- Чем они отличаются от базового Vibe Coding workflow
- Анализ существующего проекта с помощью AI agent
- Debugging и refactoring через agentic workflow
- Repo instructions и контекст проекта
- Когда эти инструменты стоит использовать
- Ограничения и риски при работе с более автономными AI tools
- Сравнение Claude Code, Codex, Cursor и GitHub Copilot
- Практические примеры использования

Проектное мышление для Vibe Coding: проблема, аудитория, MVP

- Почему проектное мышление важно при работе с AI
- Problem statement
- Target audience
- User need и user pain
- Что такое MVP
- Как сократить идею до реализуемого первого релиза
- Функции ядра и второстепенные функции
- Feature prioritization
- Практические примеры использования

Проектирование приложения для Vibe Coding: user flow, screens, acceptance criteria

- User flow
- Screen map
- Feature list
- Data objects и сущности
- Acceptance criteria
- Decomposition на маленькие задачи для AI
- Prompt strategy для приложения
- Когда использовать Bolt или Lovable, а когда Cursor и GitHub Copilot
- Практические примеры использования

Создание front-end части приложения с использованием Cursor/GitHub Copilot Claude/Codex

- Выбор идеи приложения
- Создание структуры front-end проекта
- Базовые экраны приложения
- Компоненты интерфейса
- Навигация
- Формы и пользовательский ввод
- Списки, карточки, фильтры
- Empty state, loading state, error state
- Улучшение UI через AI prompts
- Проверка и итеративная доработка интерфейса
- Практические примеры использования

Back-end основы для full stack приложения

- Что такое back-end
- Роль server-side логики в приложении
- Структура простого API
- Маршруты и endpoints
- CRUD-операции
- Обработка запросов и ответов
- Валидация входных данных
- Базовая обработка ошибок
- Разделение логики по файлам
- Использование Cursor и GitHub Copilot для генерации и правки server code
- Практические примеры использования

Database layer и работа с данными

- Что такое database
- Реляционный и NoSQL-подход на базовом уровне
- Таблицы, записи, поля, связи
- CRUD на уровне данных
- Схема данных для простого приложения
- Выбор database под задачу
- Подключение back-end к database
- Seed data и тестовые данные
- Хранение пользовательских сущностей
- Практические примеры использования

Интеграция front-end, back-end и database в единое full stack приложение

- Связь интерфейса с API
- Получение данных с сервера
- Отправка данных на сервер
- Отображение данных из database
- Создание, редактирование и удаление сущностей
- Синхронизация пользовательского интерфейса и состояния
- Обработка сетевых ошибок
- Проверка сквозных пользовательских сценариев
- Итеративная доработка full stack решения через AI
- Практические примеры использования

Тестирование, отладка и рефакторинг AI-generated приложения

- Почему AI-generated code требует обязательной проверки
- Поиск ошибок в console и terminal
- Типичные ошибки front-end
- Типичные ошибки back-end
- Типичные ошибки database integration
- Debugging workflow
- Ручное тестирование пользовательских сценариев
- Базовые чек-листы качества
- Рефакторинг структуры проекта
- Улучшение читаемости кода и prompt'ов
- Практические примеры использования

Deployment, публикация и сопровождение приложения

- Что такое deployment
- Разница между local environment и production
- Подготовка проекта к публикации
- Environment variables
- Публикация front-end
- Публикация back-end
- Подключение database в опубликованной среде
- Проверка работоспособности после deploy
- Базовый monitoring и logs
- Исправление post-release ошибок
- Практические примеры использования

Экзамен и итоговый проект: создание законченного full stack приложения

- Финальная доработка итогового проекта
- Проверка front-end, back-end, database и deployment
- Подготовка README и сценария демонстрации
- Защита итогового проекта
- Экзамен по материалам курса

